



Tecnológico de Monterrey

Evidencia 1 - Actividad Integral de Conceptos
Básicos y Algoritmos Fundamentales (Evidencia
Competencia)

Estructura de Datos

Elena María Barrios Jordan

|

A01771338

19 Sep 2026

Evidencias

Con este proyecto demostré cómo podemos tener archivos con cualquier tipo de información y aprender a extraerla para posteriormente ordenarla.

A continuación mostraré las pruebas realizadas durante el desarrollo del código con objetivo de comprobar que el programa que realicé puede leer correctamente los registros de los archivos que le dimos previamente en el apartado de **/data**, ordenarlos cronológicamente utilizando diferentes algoritmos de ordenamiento y realizar búsquedas por un rango de fecha y hora la cuál es seleccionada por el usuario.

Además al realizar las pruebas, pude revisar primeramente que cada algoritmo de ordenamiento funcionara de manera correcta, ya después analicé las diferencias en sus comportamientos. Los 6 algoritmos utilizados son los siguientes:

Swap Sort

Bubble Sort

Selection Sort

Insertion Sort

Quick Sort

Merge Sort

Las pruebas nos permiten comparar los tiempos de ejecución que obtuvimos, así como observar el comportamiento de las complejidades teóricas de cada algoritmo para poder decidir la mejor opción.

En todo el proceso de escritura de mi código fui probando pequeños checkpoints con los que pude ir registrando el correcto funcionamiento de los algoritmos. Por ejemplo para observar cómo funciona el cronómetro con todos los algoritmos simultáneamente, creé copias del mismo File para que cada copia pudiera ser ordenada al mismo tiempo, además puse un verificador el cuál se comprobó que en cada caso el file estuviera siendo ordenado correctamente:

```
//creamos una copia para poder medir los tiempos simultaneamente probando todas los algoritmos
```

```
vector<Log> logsInsertion = logs;  
vector<Log> logsQuick = logs;  
vector<Log> logsMerge = logs;  
vector<Log> logsBubble = logs;  
vector<Log> logsSelection = logs;  
vector<Log> logsSwap = logs;
```

```
clock_t start = clock();  
insertionSort(logsInsertion);  
clock_t end = clock();  
double timeInsertion = double(end - start) / CLOCKS_PER_SEC;
```

```
start = clock();  
quickSort(logsQuick, 0, logsQuick.size() - 1);  
end = clock();  
double timeQuick = double(end - start) / CLOCKS_PER_SEC;
```

```
start = clock();  
mergeSort(logsMerge, 0, logsMerge.size() - 1);  
end = clock();  
double timeMerge = double(end - start) / CLOCKS_PER_SEC;
```

nces > Evidence1 > src > main.cpp > main()

```
int main() {  
  
    if (isSorted(logsQuick)) {  
        cout << "Quick Sort: correctly sorted\n";  
    } else {  
        cout << "Quick Sort: ERROR\n";  
    }  
  
    if (isSorted(logsMerge)) {  
        cout << "Merge Sort: correctly sorted\n";  
    } else {  
        cout << "Merge Sort: ERROR\n";  
    }  
  
    if (isSorted(logsBubble)) {  
        cout << "Bubble Sort: correctly sorted\n";  
    } else {  
        cout << "Bubble Sort: ERROR\n";  
    }  
  
    if (isSorted(logsSelection)) {  
        cout << "Selection Sort: correctly sorted\n";  
    } else {  
        cout << "Selection Sort: ERROR\n";  
    }  
  
    if (isSorted(logsSwap)) {  
        cout << "Swap Sort: correctly sorted\n";  
    } else {  
        cout << "Swap Sort: ERROR\n";  
    }  
}
```

Así obtuve los siguientes resultados:

```
Insertion Sort time: 0.00933 seconds
Quick Sort time: 1.85624 seconds
Merge Sort time: 0.022613 seconds
Bubble Sort time: 0.006888 seconds
Selection Sort time: 0.12265 seconds
Swap Sort time: 0.124909 seconds
Insertion Sort: correctly sorted
Quick Sort: correctly sorted
Merge Sort: correctly sorted
Bubble Sort: correctly sorted
Selection Sort: correctly sorted
Swap Sort: correctly sorted
bjordan_em@Elenas-MacBook-Pro-8 src %
```

y separando por archivo obtuve:

File Selected: log607-1.txt

```
Insertion Sort time: 3.42639 seconds
Quick Sort time: 0.017948 seconds
Merge Sort time: 0.023284 seconds
Bubble Sort time: 3.75157 seconds
Selection Sort time: 0.194748 seconds
Swap Sort time: 3.78416 seconds
bjordan_em@Elenas-MacBook-Pro-8 src %
```

File Selected: log607-2.txt

```
Insertion Sort time: 0.009209 seconds
Quick Sort time: 1.84939 seconds
Merge Sort time: 0.022524 seconds
Bubble Sort time: 0.006949 seconds
Selection Sort time: 0.12366 seconds
Swap Sort time: 0.125966 seconds
bjordan_em@Elenas-MacBook-Pro-8 src %
```

O para mayor comprensión lo convertiré en tabla:

Archivo	Algoritmo	Tiempo
log607-1.txt	Insertion Sort	3.42639 s
log607-1.txt	Quick Sort	0.017948 s
log607-1.txt	Merge Sort	0.023284 s
log607-1.txt	Bubble Sort	3.75157 s
log607-1.txt	Selection Sort	0.194748 s

log607-1.txt	Swap Sort	3.78416 s
log607-2.txt	Insertion Sort	0.009209 s
log607-2.txt	Quick Sort	1.84939 s
log607-2.txt	Merge Sort	0.022524 s
log607-2.txt	Bubble Sort	0.006949 s
log607-2.txt	Selection Sort	0.123660 s
log607-2.txt	Swap Sort	0.125966 s

Complejidad en cada caso:

Algoritmo	Mejor caso	Peor caso
Insertion Sort	$O(n)$	$O(n^2)$
Quick Sort	$O(n \log n)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$
Bubble Sort	$O(n)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$
Swap Sort	$O(n^2)$	$O(n^2)$

Búsqueda Binaria

Ya una vez con los registros ordenados, realicé algunas pruebas para comprobar que la búsqueda por rango funcionara correctamente.

En mi código implementé dos funciones:

startBinarySearch: encuentra la primera posición que su timestamp sea mayor o igual al inicio del rango.

endBinarySearch: encuentra la primera posición la cual su timestamp es mayor que el final del rango deseado.

Así esperamos el rango que corresponde a $\text{fecha inicial} \leq \text{registro} \leq \text{fecha final}$

En un caso dónde el usuario seleccionó un rango en el que no existen registros el programa responde con:

No records were found in this range

Timestamps Duplicados

También con las pruebas realizadas, se verifica que la búsqueda por rango funcione después del ordenamiento, incluyendo casos en los que existen varios registros con la fecha y tiempo repetidos como el siguiente ejemplo dónde los dos registros ocurren el el mismo momento:

2024 10 2 23:4:24 10.14.121.223 Changes in File Integrity
2024 10 2 23:4:24 10.14.47.219 Social engineering attacks

Estos archivos corresponden a *log607-2.txt* y aunque la fecha y hora son iguales, los registros son diferentes porque tienen distinta dirección IP y diferente mensaje.

Para realizar la prueba con los registros ya mostrados se utilizó:

Fecha inicial: 2024-10-02 23:04:24

Fecha final: 2024-10-02 23:04:24

Esto significa que si nuestro programa funciona bien se deben encontrar TODOS los registros que hayan ocurrido en esa hora específica.

Esto se cumple por la condición que se cumple en la búsqueda de fechas: $\text{inicio} \leq \text{timestamp} \leq \text{final}$

Al iniciar la función de búsqueda binaria que creamos *startBinarySearch*, se encarga de buscar la primera posición que contiene el timestamp solicitado por el usuario. Esto permite encontrar el inicio del grupo de registros duplicados, posteriormente *endBinarySearch* busca la primera posición después del grupo de registros que todavía cumplen con el límite final o sea el primer registro que ya no entra en lo solicitado.

Nuestro programa muestra:

2024-10-2 23:4:24 10.14.121.223 Changes in File Integrity
2024-10-2 23:4:24 10.14.47.219 Social engineering attacks

Por lo que confirmamos que ambos registros ocurridos en el momento 2024-10-02 23:04:24 fueron contemplados en la búsqueda.

Así mismo están las evidencias de mis commits dónde fui avanzando día con día hasta el día de hoy cuando terminé con el código.

La verdad siento que lo hice un poco lento pero distribuí mis tiempos de tal manera que yo pudiera estar segura en cada parte del código que incluí además de entenderlo perfectamente.

Commits on Sep 15, 2026

main.cpp Take time done ElenaaJordan committed 18 hours ago	8658895			
Ordena los primeros 5 con insertion sort ElenaaJordan committed 18 hours ago	d385770			

Commits on Sep 14, 2026

main.cpp insertionSort ElenaaJordan committed yesterday	5f5423d			
total logs ElenaaJordan committed yesterday	ba752ef			
update ElenaaJordan committed 2 days ago	f251253			
define LOG y convierto meses ElenaaJordan committed 2 days ago	c17242f			
estructutra acabada y datooos ElenaaJordan committed 2 days ago	1e821c7			
structure ElenaaJordan committed 2 days ago	1d4c91c			

Commits on Sep 11, 2026

evidences ElenaaJordan committed 5 days ago	a6e8c10			
---	---------	--	--	--

Commits on Sep 19, 2026		
main.cpp DO WHILE	ElenaaJordan committed yesterday	c31ec06   
Commits on Sep 18, 2026		
main.cpp OUTPUT SORT FILE	ElenaaJordan committed 2 days ago	23b36ed   
main.cpp Search by date	ElenaaJordan committed 2 days ago	a5f2c0b   
main.cpp Sorting results	ElenaaJordan committed 2 days ago	a2b0e70   
Commits on Sep 17, 2026		
mai.cpp Sorting predictions and couts	ElenaaJordan committed 3 days ago	7273fd7   
main.cpp Chose sorting algorithm	ElenaaJordan committed 3 days ago	60c9375   
Commits on Sep 16, 2026		
main.cpp 6 algorithms and their time	ElenaaJordan committed 4 days ago	498de00   
main.cpp MergeSort	ElenaaJordan committed 4 days ago	c86665c   
main.cpp Toma teimpos simultaneamente (quick e insertion)	ElenaaJordan committed 4 days ago	ef90dc2   
main.cpp Qick Sort	ElenaaJordan committed 4 days ago	cb679ad   

Conclusiones

Gracias a todas las pruebas que realicé pude comprobar que el programa puede leer, ordenar y buscar registros de fecha y hora correctamente.

Probamos seis algoritmos de ordenamiento con los dos archivos que le dimos, obteniendo 12 combinaciones de ordenamientos distintas. Los resultados mostraron varias diferencias dependiendo de las características de los datos de entrada y los explicaré más adelante.

En el primer archivo *log607-1.txt* el cual contiene registros totalmente desordenados, los algoritmos con complejidad cuadrática $O(n^2)$ se ven tiempos mayores que Quick Sort y Merge Sort. Esto es consistente con los grados de complejidades teóricas que esperaríamos.

En el segundo, *log607-2.txt* contiene registros un poco más ordenados, Insertion Sort y Bubble Sort encontramos tiempos mucho menores que en el primer archivo. Esto coincide con el comportamiento esperado de estos algoritmos cuando los datos tienen un orden.

Ahora, podemos ver que Merge Sort mantuvo tiempos muy parecidos en ambos archivos y se puede explicar con su complejidad de $O(n \log n)$ tanto en el mejor como en el peor caso.

Quick Sort es el que más diferencia encontró entre los dos archivos. Con esto entendemos que su tiempo de ejecución depende mucho de cómo se distribuyen los datos a diferencia de los otros algoritmos..

También como expliqué previamente, en mi programa se comprobó la búsqueda binaria por rango después de ordenar y pudimos observar que una predicción de tiempo depende de dos cosas: Del algoritmo que usemos y de las características/Orden inicial de los datos.